Practicum Inverse Functions., Composition of Functions., Some Special Functions., Recursive Functions

WEEK 12

Subject:

Computational Mathematics

Prepared by:

Aisyah Currents

25091397108

Class International/Number 2

INFORMATICS MANAGEMENT STUDY PROGRAM

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2025

Table of Contents

PRACTICUM Part 1: Applications of Matrices., Relations., Representation of Relations with Tables., Matrices., and with Directed Graphs.

1. Matrices

A matrix is a collection of numbers arranged in rows and columns. A matrix is an arrangement of numbers in a rectangular form, consisting of rows and columns. Matrices are used to store data and perform various operations such as addition, multiplication, and transpose. Matrices are widely used in various fields such as imaging and data science. An example of a 2x3 matrix application is representing product sales numbers over two days in three regions.

The Source Code with output

```
Kuliah > Matkom Praktikum 3 > part1.py > ...
1  # Number 1
2  import numpy as np
3
4  sales = np.array([
5      [100, 150, 200],
6      [80, 120, 160]
7  ])
8
9  total_sales_per_region = np.sum(sales, axis=0)
10 print("Sales matrix:")
11 print(sales)
12 print("Total sales per region:", total_sales_per_region)
```

```
ipts/python.exe "c:/Users/USER/OneDrive/Documents/Currents/Pyth
Sales matrix:
[[100 150 200]
 [ 80 120 160]]
Total sales per region: [180 270 360]
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

The code loads NumPy and defines a 2×3 array named *sales*, representing sales data from two regions across three product categories. It then calculates the total sales per region by summing the columns using np.sum (sales, axis=0), producing the combined totals for each product across both regions. The script prints the original sales matrix and the computed totals, making it easy to observe how the data is structured and how NumPy's array operations simplify aggregated calculations.

2. Relations

A relation is a connection between two elements in one or more sets. An example of a relation is "greater than" in a set of numbers. Given the set A={1,2,3}, determine the relation "less than" on that set.

The Source Code with output

```
14  # Number 2
15  A = {1, 2, 3}
16
17  relation = [(a, b) for a in A for b in A if a < b]
18  print("Pairs in the 'less than' relation:", relation)
```

```
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
ipts/python.exe "c:/Users/USER/OneDrive/Documents/Currents/Py
Pairs in the 'less than' relation: [(1, 2), (1, 3), (2, 3)]
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

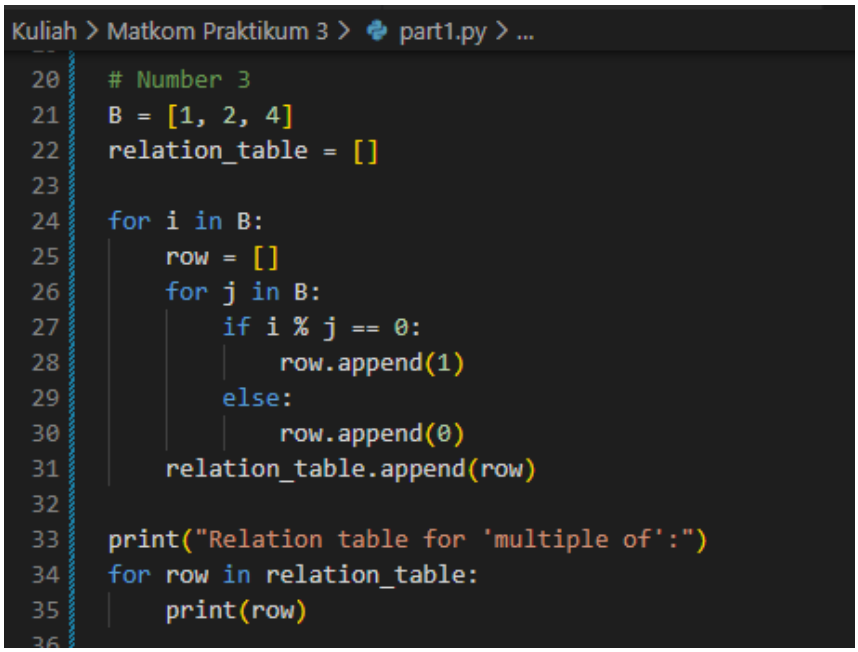
The Analysis

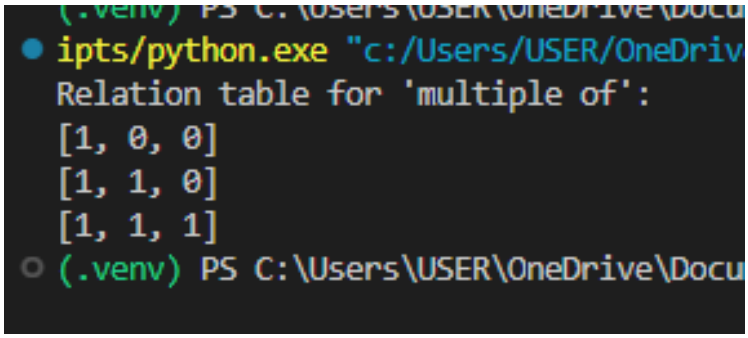
This code defines a set A = {1,2,3} and then uses a list comprehension to generate all ordered pairs (a,b) where both elements come from A and satisfy the condition a<b. This effectively constructs the “less than” relation on the set, producing all pairs where the first number is smaller than the second. The resulting list [(1,2), (1,3), (2,3)] is then printed, clearly showing how the relation captures the natural ordering within the set.

3. Representation of Relations with Tables

A table representation is a simple way to display the relation between two elements in the form of a binary table (0 and 1). For example, consider the relation "multiple of" on the set B = {1,2,4}. Create a table that shows whether an element is a multiple of another element.

The Source Code with output





The Analysis

The code builds a relation table for the “multiple of” relation on the set B = [1, 2, 4] by checking, for each pair (i, j), whether i is a multiple of j. It constructs this table row by row: for each element i in B, it loops through every j in B and appends 1 to the row if i % j == 0 (meaning j divides i), otherwise it appends 0. Each completed row is added to relation_table, producing a matrix that visually represents which elements of B are multiples of which others, and the final loop prints the table one row at a time.

4. Representation of Relations with Matrices

A matrix can be used to represent a relation, where each element is filled with 1 or 0 according to the relationship between the elements. Display the relation "greater than" on the set C={1,2,3} in matrix form.

The Source Code with output

```python
# Number 4
C = [1, 2, 3]
relation_matrix = []

for i in C:
    row = []
    for j in C:
        if i > j:
            row.append(1)
        else:
            row.append(0)
    relation_matrix.append(row)

print("Matrix for the 'greater than' relation:")
for row in relation_matrix:
    print(row)
```

```
PS C:\Users\USER\OneDrive\Documents\Current
ivate.ps1
(.venv) PS C:\Users\USER\OneDrive\Documents
ipts/python.exe "c:/Users/USER/OneDrive/Doc
Matrix for the 'greater than' relation:
[0, 0, 0]
[1, 0, 0]
[1, 1, 0]
(.venv) PS C:\Users\USER\OneDrive\Documents
```

The Analysis

This code constructs a relation matrix for the “greater than” relation on the set C = [1, 2, 3] by checking each ordered pair (i, j) and marking a 1 when i is greater than j, otherwise marking a 0. For every element i in C, it creates a row and fills it by comparing i with each j in C, then appends the completed row to relation_matrix, forming a 3×3 matrix that visually represents which elements are greater than others, and finally prints the matrix row by row.

5. **Representation of Relations with Directed Graphs.**

A directed graph is a visual way to represent relations between elements in a set, using nodes and arrows that show the direction of the relation.

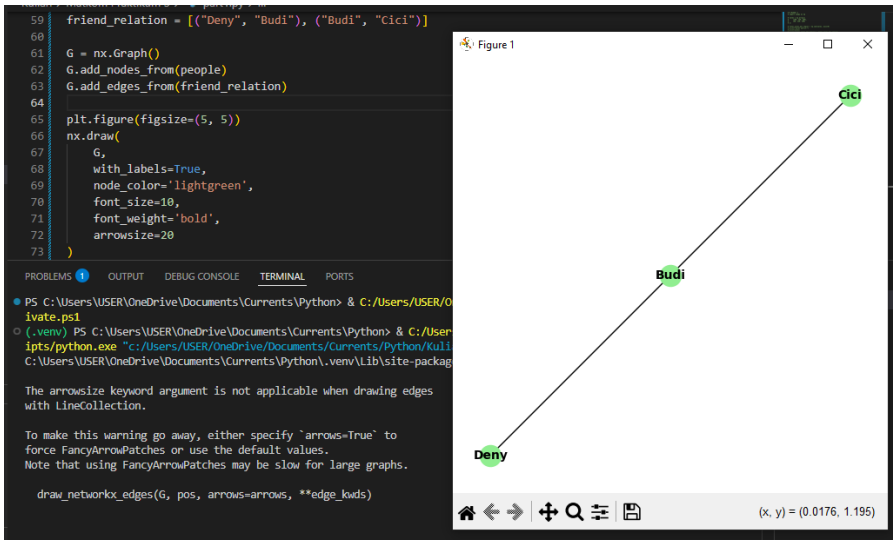
Display the "friend" relation on the set of people D={"Deny","Budi","Cici"} with the following relations:

• Deny is friends with Budi.

• Budi is friends with Cici.

The Source Code with output

```python
# Number 5
import networkx as nx
import matplotlib.pyplot as plt

people = ["Deny", "Budi", "Cici"]
friend_relation = [("Deny", "Budi"), ("Budi", "Cici")]

G = nx.Graph()
G.add_nodes_from(people)
G.add_edges_from(friend_relation)

plt.figure(figsize=(5, 5))
nx.draw(
    G,
    with_labels=True,
    node_color='lightgreen',
    font_size=10,
    font_weight='bold',
    arrowsize=20
)
plt.title("Directed graph for the 'Friend' relation")
plt.show()
```



The Analysis

This code visualizes a “friend” relation using NetworkX and Matplotlib by creating a graph whose nodes are the names in the people list and whose edges come from the friend_relation pairs. After building the graph with add_nodes_from and add_edges_from, it draws the network with labels, customized colors, and styling

so the connections are easy to see. The final plot shows a simple directed graph illustrating who is connected to whom in the defined friendship pairs.

CONCLUSIONS

1. Matrices can be used to represent data in the form of rows and columns, and they are used for various calculations.
2. Relations can be represented using tables and matrices, which help in analyzing the relationships between elements.
3. Directed graphs provide a visualization of relations, making it easier to understand patterns of relationships between elements.

PRACTICUM Part 2: Case studies of Matrices., Relations., Representation of Relations with Tables., Matrices., and with Directed Graphs.

1. Matrices

A matrix is an arrangement of numbers in a rectangular form, consisting of rows and columns. Matrices are used to store data and perform various operations such as addition, multiplication, and transpose.

•Create a 3×3 matrix and display its elements.

•Perform addition and multiplication operations on two 2×2 matrices.

•Implement a function to find the transpose of a matrix.

The Source Code with output

Kuliah > Matkom Praktikum 3 > part2.py > create_relation_table

```python
# Number 1
import numpy as np

matrix_a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Matrix A:\n", matrix_a)

matrix_b = np.array([[1, 1], [2, 2]])
matrix_c = np.array([[3, 3], [4, 4]])
result_add = matrix_b + matrix_c
print("\nMatrix Addition (B + C):\n", result_add)

transpose_a = np.transpose(matrix_a)
print("\nTranspose of Matrix A:\n", transpose_a)

```

```
ipts/python.exe "c:/Users/USE
Matrix A:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]

Matrix Addition (B + C):
 [[4 4]
 [6 6]]

Transpose of Matrix A:
 [[1 4 7]
 [2 5 8]
 [3 6 9]]
```

The Analysis

This code uses NumPy to demonstrate basic matrix operations: it first creates a 3×3 matrix matrix_a and prints it, then defines two smaller matrices, matrix_b and matrix_c, adds them together using NumPy's element-wise addition, and displays the result. After that, it computes the transpose of matrix_a with np.transpose, which flips the matrix over its diagonal, turning rows into columns, and prints the transposed version, making the example a clear illustration of matrix creation, addition, and transposition.

2. Relations

A relation is a rule that connects two sets. A relation can be interpreted as pairs of elements between two sets. For example, sets A and B have a relation R if there is a rule that connects elements of A to elements of B.

•Define set A = {1, 2, 3} and set B = {2, 4, 6}. Determine all ordered pairs (a, b) that satisfy the relation "a is a factor of b."

•Create a Python function to check whether the relation is reflexive, symmetric, or transitive.

The Source Code with output

```python

# Number 2
A = {1, 2, 3}
B = {2, 4, 6}

relation = [(a, b) for a in A for b in B if b % a == 0]
print("Relation R:\n", relation)

def is_reflexive(relation, set_a):
    return all((a, a) in relation for a in set_a)

print("Is the relation reflexive?", is_reflexive(relation, A))

```

```
ipts/python.exe "c:/Users/USER/OneDrive/Documents/Currents/Pytho
Relation R:
 [(1, 2), (1, 4), (1, 6), (2, 2), (2, 4), (2, 6), (3, 6)]
Is the relation reflexive? False
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

This code builds a relation R between sets A = {1, 2, 3} and B = {2, 4, 6} by including each pair (a, b) where b is divisible by a. After printing the resulting list of pairs, it defines a function is_reflexive that checks whether every element a in set A has the pair (a, a) inside the relation. Since the relation is formed based on divisibility with elements of B, the relation does not contain any (a, a) pairs, so the reflexive check will return False. Overall, the code shows how to construct a divisibility-based relation and test whether it satisfies the reflexive property.

3. Representation of Relations with Tables

Representation of a relation using a table is a way to illustrate the relation between two sets in table form, where rows and columns represent the elements of the sets. If the relation holds between elements, the table is marked (for example, 1 to indicate the relation exists, 0 if it does not).

• Create a relation table for sets A = {1, 2, 3} and B = {2, 4, 6} based on the relation "a is a factor of b."

• Implement a function that displays the relation table.

The Source Code with output

```
Kuliah > Matkom Praktikum 3 > part2.py > ...
27  # Number 3
28  def create_relation_table(set_a, set_b, relation):
29      table = [
30          [1 if (a, b) in relation else 0 for b in set_b]
31          for a in set_a
32      ]
33      return table
34
35  table = create_relation_table(A, B, relation)
36  for row in table:
37      print(row)
38
```

```
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> &
ipts/python.exe "c:/Users/USER/OneDrive/Documents/Currents/Pyth
Relation R:
 [(1, 2), (1, 4), (1, 6), (2, 2), (2, 4), (2, 6), (3, 6)]
[1, 1, 1]
[1, 1, 1]
[0, 0, 1]
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

This code defines a function that generates a relation table by checking, for every pair (a,b) from two sets, whether the pair exists in a given relation and placing a 1 if it does or a 0 if it does not, producing a matrix where each row corresponds to an element of set A and each column corresponds to an element of set B, and finally prints the resulting table as a clear visual representation of the relation.

4. Representation of Relations with Matrices

Representation of a relation using a matrix is another way to illustrate a relation in the form of a binary matrix. The matrix elements have a value of 1 if the relation holds between the row and column elements, and 0 if it does not.

• Represent the relation “a is a factor of b” between sets A = {1, 2, 3} and B = {2, 4, 6} in matrix form.

• Implement a function to display the relation matrix.

The Source Code with output

```python
# Number 4
def create_relation_matrix(set_a, set_b, relation):
    matrix = [
        [1 if (a, b) in relation else 0 for b in set_b]
        for a in set_a
    ]
    return np.array(matrix)

matrix = create_relation_matrix(A, B, relation)
print("Relation Matrix:\n", matrix)

```

```
ipts/python.exe "c:/Users/USER/OneDrive/Documents/Currents/Pytho
Relation R:
 [(1, 2), (1, 4), (1, 6), (2, 2), (2, 4), (2, 6), (3, 6)]
Relation Matrix:
 [[1 1 1]
 [1 1 1]
 [0 0 1]]
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

The code generates a relation matrix by checking each pair ((a, b)) from two sets and marking the entry with 1 if the pair exists in the given relation, or **0** if it does not, effectively converting an abstract relation into a structured numerical form. This is done using a nested list comprehension that iterates over all elements of set_a as rows and all elements of set_b as columns, ensuring that every possible ordered pair is evaluated. The resulting list of lists is then converted into a NumPy array for easier manipulation and readability. Overall, this function provides a clear and efficient way to visualize relationships between sets in matrix form, which is especially useful for analyzing mathematical relations, graph representations, or binary relations in discrete mathematics.

5. Representation of Relations with Directed Graphs.

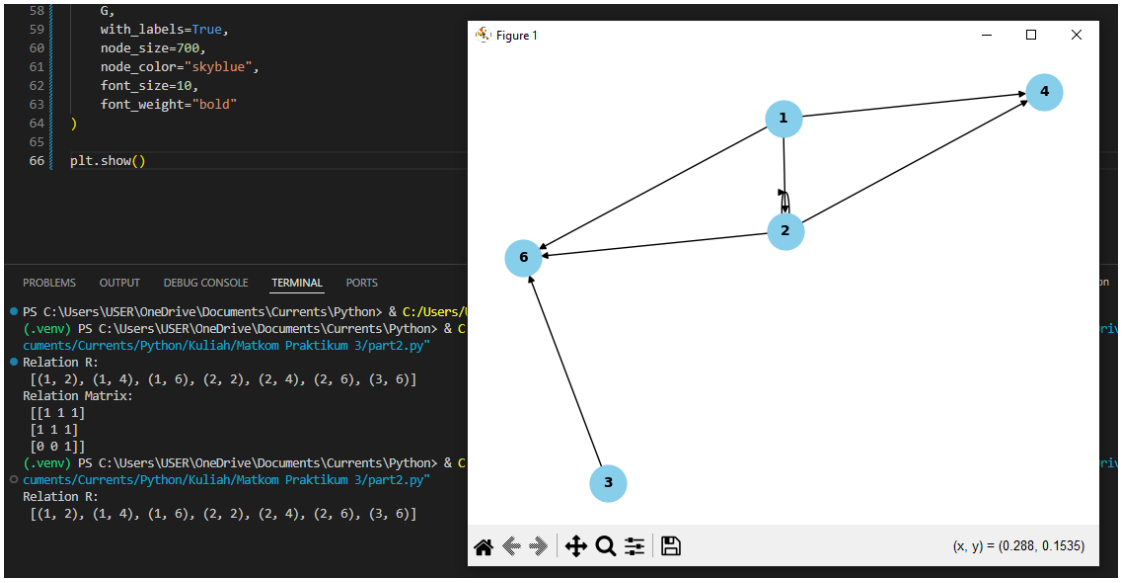
A relation can also be represented using a directed graph, where the elements of the sets are represented as nodes, and the relation between elements is represented as arrows connecting the nodes.

• Draw a directed graph for the relation “a is a factor of b” between A = {1, 2, 3} and B = {2, 4, 6}.

• Use the *networkx* library to draw the directed graph.

The Source Code with output

```python
# Number 5
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()
G.add_edges_from(relation)

nx.draw(
    G,
    with_labels=True,
    node_size=700,
    node_color="skyblue",
    font_size=10,
    font_weight="bold"
)

plt.show()
```



The Analysis

The code constructs a directed graph using NetworkX to visually represent a relation between elements by interpreting each ordered pair in the relation as a directed edge in the graph. After creating a DiGraph object and adding all edges using add_edges_from(relation), the program uses nx.draw() with styling options such as sky-blue node color, bold labels, and a fixed node size to produce a clear and readable visualization of the relational structure. This graphical representation helps illustrate how elements are connected directionally, making it easier to analyze properties of the relation such as connectivity, loops, symmetry, or transitivity compared to viewing the relation only as a list of pairs or a matrix.

Finally, plt.show() displays the graph, allowing the user to visually interpret and evaluate the underlying relational structure.

CONCLUSIONS

In this module, we have learned various ways to represent relations between sets using tables, matrices, and directed graphs. These approaches help in better understanding relations and enable various applications in the field of computing.

PRACTICUM Part 3: Practice Exercises of Matrices., Relations., Representation of Relations with Tables., Matrices., and with Directed Graphs.

1. Matrices

Suppose there are two matrices:

A = [[1, 2], [3, 4]]

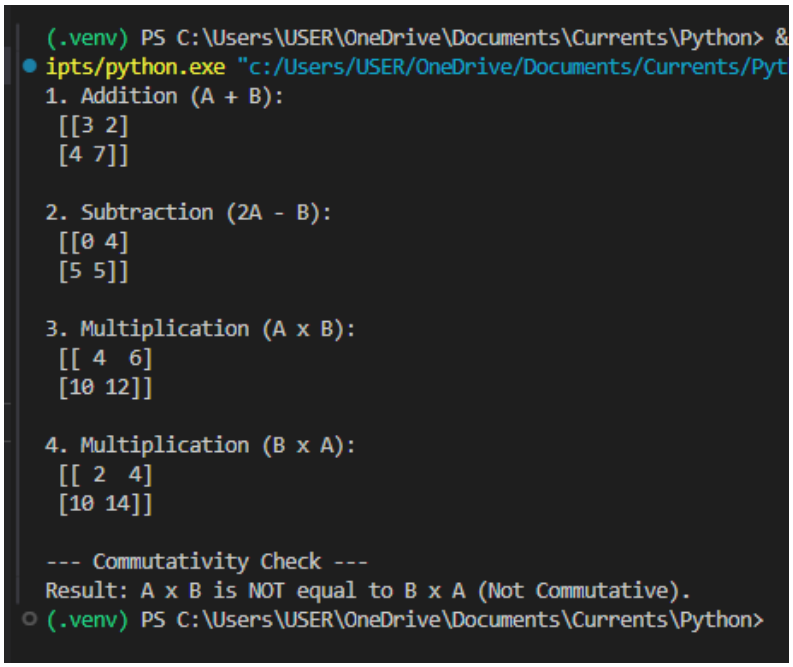
B = [[2, 0], [1, 3]]

Compute the following operations using Python:
• Addition (A + B)
• Subtraction (2A − B)
• Multiplication (A × B) and (B × A)

Practicum Task Steps:
• Create a Python function matrix_operations(A, B) to compute the matrix operations above.
• Use the NumPy library to simplify the calculations.
• Display the result of each operation.
• Check whether A × B = B × A.
• Add explanatory comments to the code.

The Source Code with output

```python
# Number 1
import numpy as np

def matrix_operations(A, B):
    """
    Function to perform addition, subtraction, and multiplication
    on two given matrices using NumPy.
    """

    # --- 1. Addition (A + B) ---
    # NumPy allows element-wise addition directly with the + operator
    addition_result = A + B
    print("1. Addition (A + B):\n", addition_result)

    # --- 2. Subtraction (2A - B) ---
    # First multiply matrix A by scalar 2, then subtract matrix B
    subtraction_result = (2 * A) - B
    print("\n2. Subtraction (2A - B):\n", subtraction_result)

    # --- 3. Multiplication (A x B) ---
    # Use the @ operator or np.dot() for matrix multiplication
    mul_AB = A @ B
    print("\n3. Multiplication (A x B):\n", mul_AB)

    # --- 4. Multiplication (B x A) ---
    mul_BA = B @ A
    print("\n4. Multiplication (B x A):\n", mul_BA)

    # --- 5. Check Commutativity (A x B = B x A) ---
    # np.array_equal checks if both arrays contain the exact same elements
    is_equal = np.array_equal(mul_AB, mul_BA)

    print("\n--- Commutativity Check ---")
    if is_equal:
        print("Result: A x B is equal to B x A (Commutative).")
    else:
        print("Result: A x B is NOT equal to B x A (Not Commutative).")

# --- Main Execution ---

# Define Matrix A and B using NumPy arrays
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[2, 0],
              [1, 3]])

# Call the function
matrix_operations(A, B)
```



2. Relations

Given two sets:

A = {1, 2, 3, 4}

B = {2, 4, 6, 8}

Relation R is defined by the rule **b = 2a**.

Practicum Task Steps:
• Create a Python function generate_relation(A, B) that produces all pairs (a, b) that satisfy **b = 2a**.
• Display the result of the relation in the form of a list of ordered pairs.
• Also test the relation for **b = a²**.
• Add comments to each part of the code.

The Source Code with output

```python

# Number 2
def generate_relation(A, B):
    """
    Generate all ordered pairs (a, b) from sets A and B
    that satisfy a given rule.
    """

    # --- Relation 1: b = 2a ---
    relation_double = []  # list to store pairs satisfying b = 2a
    for a in A:
        for b in B:
            if b == 2 * a:      # check relation rule
                relation_double.append((a, b))

    print("Relation R (b = 2a):", relation_double)

    # --- Relation 2: b = a^2 ---
    relation_square = []  # list to store pairs satisfying b = a^2
    for a in A:
        for b in B:
            if b == a**2:      # check square relation rule
                relation_square.append((a, b))

    print("Relation for b = a^2:", relation_square)


# --- Test Data ---
A = {1, 2, 3, 4}
B = {2, 4, 6, 8}

# --- Run the function ---
generate_relation(A, B)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users/USER/
ate.ps1
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Use
ts/python.exe "c:/Users/USER/OneDrive/Documents/Currents/Python/Kulia
Relation R (b = 2a): [(1, 2), (2, 4), (3, 6), (4, 8)]
Relation for b = a^2: [(2, 4)]
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

3. Representation of Relations with Tables

Use the relation **R = {(1,2), (2,4), (3,6), (4,8)}**.

Practicum Task Steps:

• Create a Python function relation_table(A, B, R) to display the relation in table form.

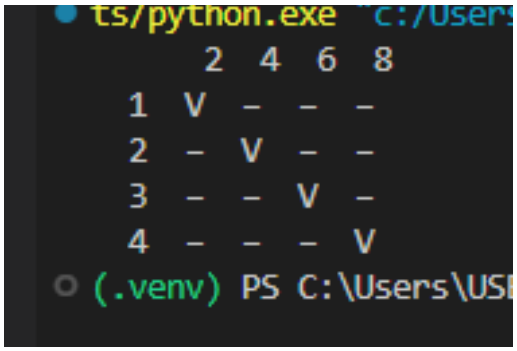
• Use the symbol ✓ if a relation exists and – if it does not.

• Use nested loops to print the table.

• Add comments to the code to explain the logic.

The Source Code with output

```python
# Number 3
def relation_table(A, B, R):
    """
    Display the relation R in table form.
    Uses V if (a, b) is in R and - if not.
    """

    # Convert sets to sorted lists for consistent table order
    A = sorted(A)
    B = sorted(B)

    # Print header row
    print("    ", end="")
    for b in B:
        print(f"{b:>3}", end="")
    print()

    # Print each row for elements of A
    for a in A:
        print(f"{a:>3} ", end="")
        for b in B:
            # Check whether the pair (a, b) is in relation R
            if (a, b) in R:
                print(" V ", end="")
            else:
                print(" - ", end="")
        print()


# --- Given Data ---
A = {1, 2, 3, 4}
B = {2, 4, 6, 8}
R = {(1,2), (2,4), (3,6), (4,8)}

# Run the function
relation_table(A, B, R)
```



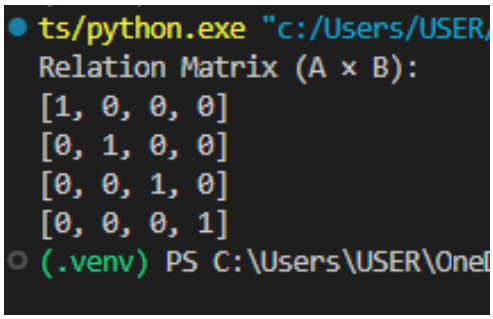
4. Representation of Relations with Matrices

Use the relation **R = {(1,2), (2,4), (3,6), (4,8)}**.

Practicum Task Steps:
• Create a Python function relation_matrix(A, B, R) to form the binary relation matrix.
• Use the number **1** if the pair (a, b) is in the relation and **0** if it is not.
• Display the result in the form of a 2-dimensional list.
• Add brief comments for each step of the code.

The Source Code with output

```python

# Number 4
def relation_matrix(A, B, R):
    """
    Create and display the binary matrix for relation R
    using 1 if (a, b) is in R and 0 otherwise.
    """

    # Sort sets for consistent matrix order
    A = sorted(A)
    B = sorted(B)

    # Initialize an empty 2-dimensional list (matrix)
    matrix = []

    # Loop over every element a in A (rows)
    for a in A:
        row = []
        # Loop over every element b in B (columns)
        for b in B:
            # Append 1 if (a, b) is in the relation, else 0
            row.append(1 if (a, b) in R else 0)
        matrix.append(row)

    # Display the resulting matrix
    print("Relation Matrix (A × B):")
    for row in matrix:
        print(row)

    return matrix


# Given data
A = {1, 2, 3, 4}
B = {2, 4, 6, 8}
R = {(1,2), (2,4), (3,6), (4,8)}

# Run the function
relation_matrix(A, B, R)
```



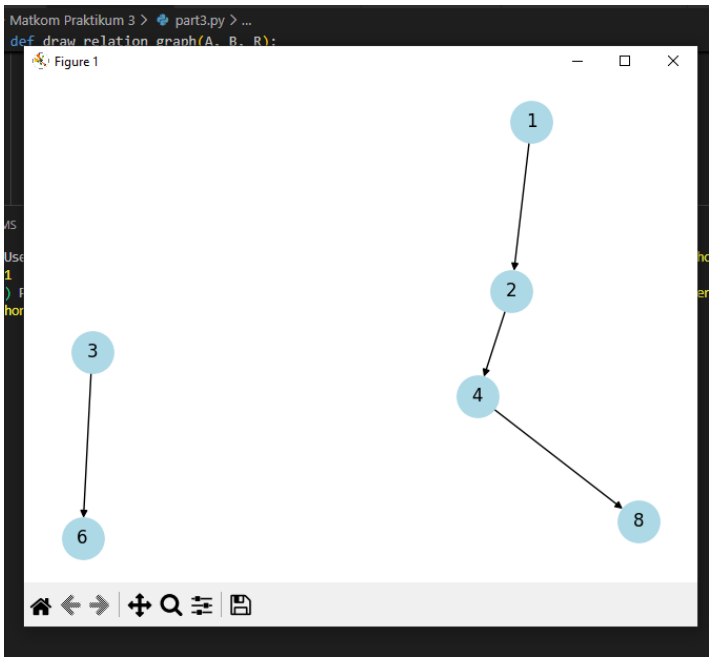
5. Representation of Relations with Directed Graphs.

Use the relation **R = {(1,2), (2,4), (3,6), (4,8)}**.

Practicum Task Steps:
• Use the *networkx* and *matplotlib* libraries.
• Create a Python function draw_relation_graph(A, B, R) to display the directed graph.
• Add nodes from A and B, then connect each pair (a, b) with an arrow.
• Display the resulting graph using nx.draw().
• Add a title to the graph and label the nodes.

The Source Code with output

```python

# Number 5
import networkx as nx
import matplotlib.pyplot as plt

def draw_relation_graph(A, B, R):
    """
    Draw a directed graph for relation R using networkx and matplotlib.
    Nodes come from sets A and B, and edges follow the ordered pairs in R.
    """

    # Create a directed graph object
    G = nx.DiGraph()

    # Add nodes from A and B
    G.add_nodes_from(A)
    G.add_nodes_from(B)

    # Add directed edges based on relation R
    for (a, b) in R:
        G.add_edge(a, b)

    # Create a layout for positioning nodes visually
    pos = nx.spring_layout(G)

    # Draw the nodes, edges, and labels
    nx.draw(G, pos, with_labels=True, arrows=True, node_color="lightblue", node_size=800)

    # Add a title
    plt.title("Directed Graph of Relation R")

    # Show the graph
    plt.show()


# Given data
A = {1, 2, 3, 4}
B = {2, 4, 6, 8}
R = {(1,2), (2,4), (3,6), (4,8)}

# Run the function
draw_relation_graph(A, B, R)
```



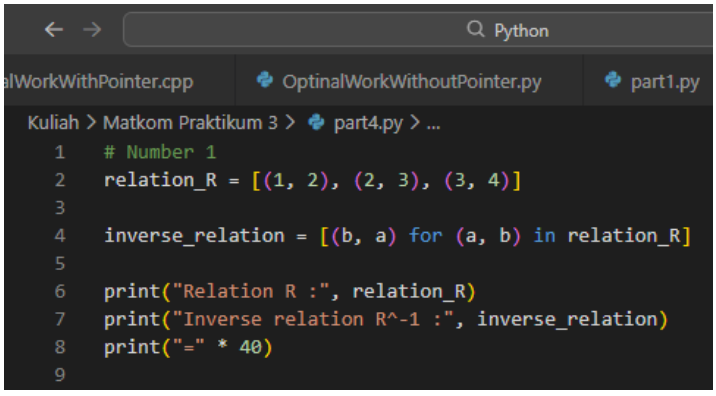
PRACTICUM Part 4: Applications of Inverse Relations., Combining Relations., Composition of Relations., Properties of Relations., Equivalence Relations.

1. Inverse Relations

The inverse relation of a relation R from set A to B is the set of reversed pairs from R. If R = {(a, b)}, then the inverse R^{-1} = {(b, a)}.

Suppose there is a relation R = {(1,2), (2,3), (3,4)}. Find its inverse relation.

The Source Code with output



```
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Praktik
Relation R : [(1, 2), (2, 3), (3, 4)]
Inverse relation R^-1 : [(2, 1), (3, 2), (4, 3)]
========================================
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

The code defines a relation R as a list of ordered pairs and then computes its inverse relation by swapping each pair (a,b) into (b,a) using a list comprehension. This effectively reverses the direction of every ordered pair, transforming the relation into its inverse R^{-1}, which is useful for analyzing how elements relate when the roles of domain and codomain are switched. The program then prints both the original relation and its inverse, separated by a visual divider line, allowing the user to clearly compare the structure of the relation before and after inversion. Overall, the code provides a simple and efficient demonstration of how inverse relations can be generated programmatically in Python.

2. Combining Relations

The combination of two relations R and S from the same set is the union of the elements of both relations. Suppose R = {(1,2), (2,3)} and S = {(2,4), (3,5)}. Combine these two relations.

The Source Code with output

```
10  # Number 2
11  relation_R = [(1, 2), (2, 3)]
12
13  relation_S = [(2, 4), (3, 5)]
14
15  combined_relation = relation_R + relation_S
16
17  print("Relation R:", relation_R)
18  print("Relation S:", relation_S)
19  print("Combined Relation:", combined_relation)
20  print("=" * 40)
21
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users/US
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Prakt
Relation R: [(1, 2), (2, 3)]
Relation S: [(2, 4), (3, 5)]
Combined Relation: [(1, 2), (2, 3), (2, 4), (3, 5)]
========================================
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

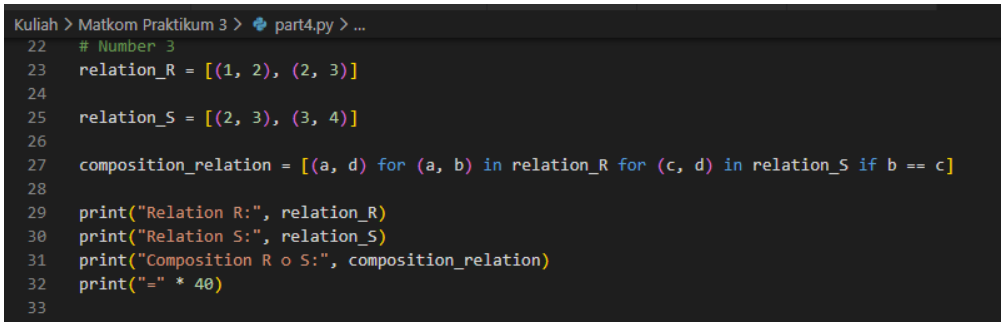
The Analysis

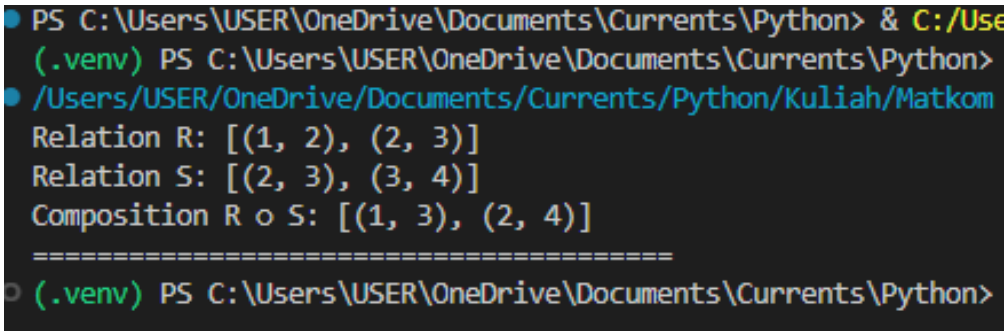
The code defines two separate relations, R and S, each represented as a list of ordered pairs, and then combines them by using the + operator, which concatenates the two lists to form a new relation containing all pairs from both. This approach effectively creates the union of the two relations without removing duplicates, which is appropriate when treating relations as simple collections rather than mathematical sets. By printing each relation individually and then displaying the combined result, the program clearly demonstrates how relations can be merged and how their ordered pairs accumulate when combined. Overall, the code provides an easy and direct method to visualize and understand how two relations can be joined in Python.

3. Composition of Relations

The composition of relations R and S involves finding elements (a,c) for which there exists an element b such that (a,b) ∈ R and (b,c) ∈ S. Suppose R = {(1,2), (2,3)} and S = {(2,3), (3,4)}. Find the composition R∘S.

The Source Code with output





The Analysis

The code performs the composition of two relations R and S by using a nested list comprehension that checks for matching middle elements between the ordered pairs of the two relations. Specifically, for every pair (a,b) in relation R and every pair (c,d) in relation S, the code selects those where b=c, meaning the output of the first relation matches the input of the second; when this condition is met, the composed pair (a,d) is added to the result. This process correctly implements the mathematical definition of relation composition R∘S, demonstrating how elements from two relations can be linked to form new relational connections. The printed output clearly shows the original relations and their composition, allowing easy comparison and deeper understanding of how the composition operation transforms relational structures.

4. Properties of Relations

Some important properties of relations are:

• Reflexive: All elements a in the set have the pair (a,a) in the relation.

• Symmetric: If (a,b) is in the relation, then (b,a) is also in the relation.

• Transitive: If (a,b) and (b,c) are in the relation, then (a,c) is also in the relation.

Check whether the relation R = {(1,1), (2,2), (1,2), (2,1)} is reflexive, symmetric, and transitive.

The Source Code with output

```python
# Number 4
relation_R = [(1, 1), (2, 2), (1, 2), (2, 1)]

set_A = {1, 2}

is_reflexive = all((a, a) in relation_R for a in set_A)

is_symmetric = all((b, a) in relation_R for (a, b) in relation_R)

is_transitive = all(
    (a, c) in relation_R
    for (a, b) in relation_R
    for (c, d) in relation_R
    if b == c
)

print("Relation R:", relation_R)
print("Reflexive:", is_reflexive)
print("Symmetric:", is_symmetric)
print("Transitive:", is_transitive)
print("=" * 40)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> &
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Pr
Relation R: [(1, 1), (2, 2), (1, 2), (2, 1)]
Reflexive: True
Symmetric: True
Transitive: True
========================================
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

The code checks whether a given relation R on the set A = {1,2} satisfies the properties of reflexivity, symmetry, and transitivity by using Python's all() function combined with list comprehensions. Reflexivity is verified by ensuring that every element aaa in the set has the pair (a,a) present in the relation, while symmetry is confirmed by checking that for every pair (a,b) in R, the reverse pair (b,a) is also included. Transitivity is evaluated by searching for all possible combinations (a,b) and (b,c) within the relation and ensuring that the implied pair (a,c) always exists; if even one required pair is missing, the relation fails transitivity. Once all three properties have been evaluated, the program prints the relation together with the Boolean results, providing a clear and structured way to determine whether the

given relation qualifies as an equivalence relation or possesses any of its individual relational properties.

5. Equivalence Relations

An equivalence relation is a relation that is reflexive, symmetric, and transitive. If a relation satisfies these three properties, then it is an equivalence relation.

Suppose there is a relation R = {(1,1), (2,2), (1,2), (2,1)}. Determine whether this relation is an equivalence relation.

The Source Code with output

```python
# Number 5
relation_R = [(1, 1), (2, 2), (1, 2), (2, 1)]

set_A = {1, 2}

is_reflexive = all((a, a) in relation_R for a in set_A)

is_symmetric = all((b, a) in relation_R for (a, b) in relation_R)

is_transitive = all(
    (a, c) in relation_R
    for (a, b) in relation_R
    for (c, d) in relation_R
    if b == c
)

is_equivalence = is_reflexive and is_symmetric and is_transitive

print("Relation R:", relation_R)
print("Is R an equivalence relation?:", is_equivalence)
print("=" * 40)
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python>
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah
Relation R: [(1, 1), (2, 2), (1, 2), (2, 1)]
Is R an equivalence relation?: True
========================================
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\
```

The Analysis

The code evaluates whether a given relation R on the set A={1,2} qualifies as an equivalence relation by checking three fundamental properties: reflexivity, symmetry, and transitivity. Reflexivity is confirmed by verifying that both (1,1) and (2,2) appear in the relation, while symmetry is validated by ensuring that for every

ordered pair (a,b) in R, its reverse (b,a) is also included. Transitivity is tested by examining all possible pair chains (a,b) and (b,c) within R and ensuring that the resulting pair (a,c) is present. After computing these Boolean results, the code determines whether the relation is an equivalence relation by combining all three conditions with logical AND. Finally, it prints the relation and a clear statement indicating whether or not it satisfies all the required properties, giving a concise summary of the relation's structural characteristics.

CONCLUSIONS

Through this lab/practical, you have learned how to apply inverse relations, combine relations, compose relations, examine properties of relations, and work with equivalence relations in Python. Each of these relation concepts forms a foundation for further study of data structures and set theory in Python.

PRACTICUM Part 5: Practice Exercises of Inverse Relations., Combining Relations., Composition of Relations., Properties of Relations., Equivalence Relations.

1. **Inverse Relations**

   Suppose there is a relation R = {(1, 2), (2, 3), (3, 4), (4, 5)}.

   Determine its inverse relation, namely R−1.

   Practical Assignment Steps:

   • Create a Python function inverse_relation(R) that swaps each pair (a, b) to (b, a).

   • Display the result of the inverse relation.

   • Add comments explaining the program steps.

   • Test the function with another relation, for example {(a, b), (b, c), (c, d)}.

   The Source Code with output

```python
# Number 1
# Function to compute the inverse of a relation
def inverse_relation(R):
    # Initialize an empty set to hold the inverse relation
    R_inverse = set()
    # Loop over each pair (a, b) in the original relation R
    for a, b in R:
        # Swap the pair to (b, a) and add to R_inverse
        R_inverse.add((b, a))
    # Return the inverse relation
    return R_inverse

# Define the original relation R
R = {(1, 2), (2, 3), (3, 4), (4, 5)}
# Calculate the inverse relation
R_inv = inverse_relation(R)
# Display the inverse relation
print("Inverse of R:", R_inv)

# Test with another relation
another_relation = {('a', 'b'), ('b', 'c'), ('c', 'd')}
# Calculate the inverse of the new relation
inverse_another_relation = inverse_relation(another_relation)
# Display the result
print("Inverse of the other relation:", inverse_another_relation)

```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users/USER/OneDrive/Document
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users/USER/OneDrive/
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Praktikum 3/part5.py"
Inverse of R: {(3, 2), (5, 4), (2, 1), (4, 3)}
Inverse of the other relation: {('b', 'a'), ('c', 'b'), ('d', 'c')}
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

2. Combining Relations

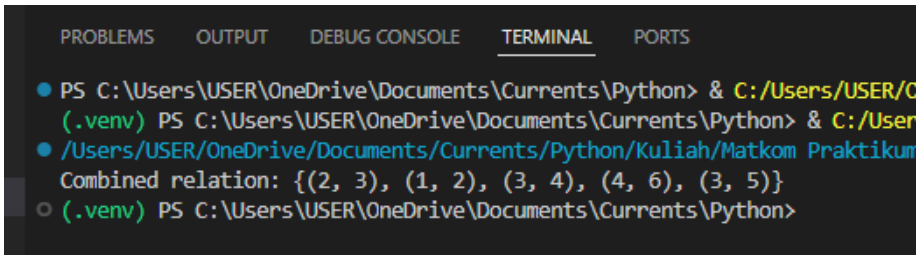
Given two relations: $R_1 = \{(1, 2), (2, 3), (3, 4)\}$, $R_2 = \{(3, 5), (4, 6)\}$. Combine these two relations into a single relation set.

Practical Assignment Steps:

- Create a Python function combine_relations(R1, R2) to combine two relations.
- Use the union operation on Python sets.
- Display the result of the combined relations.
- Add code comments to explain the combination logic.

The Source Code with output

```python

# Number 2
# Function to combine two relations R1 and R2
def combine_relations(R1, R2):
    # Use the union operator to combine the two sets of relations
    combined_relation = R1.union(R2)
    # Return the combined relation set
    return combined_relation

# Define the first relation R1
R1 = {(1, 2), (2, 3), (3, 4)}
# Define the second relation R2
R2 = {(3, 5), (4, 6)}

# Call the function to combine R1 and R2
combined_relation = combine_relations(R1, R2)
# Display the combined relation set
print("Combined relation:", combined_relation)
```



3. Composition of Relations

Given: R={(1,2),(2,3),(3,4)}, S={(2,5),(3,6),(4,7)}. Determine the result of the composition S∘R

Practical Assignment Steps:

• Create a Python function compose_relations(R, S) to find the composition of relations.

• Use the logic: (a,c)∈S∘R if there exists a b such that (a,b)∈R and (b,c)∈S.

• Display the result of the composition relation.

• Add comments to every important part of the code.

The Source Code with output

```python
# Number 3
def compose_relations(R, S):
    # Initialize an empty set to store the composed relation
    composition = set()

    # Loop through each pair (a, b) in relation R
    for (a, b) in R:
        # For each such pair, check all pairs (b2, c) in relation S
        for (b2, c) in S:
            # If the intermediate element matches (b == b2), add (a, c) to the result
            if b == b2:
                composition.add((a, c))
    # Return the composed relation as a set of pairs
    return composition

# Define relation R
R = {(1, 2), (2, 3), (3, 4)}
# Define relation S
S = {(2, 5), (3, 6), (4, 7)}

# Calculate the composition S ∘ R
S_comp_R = compose_relations(R, S)
# Display the result of the composition
print("S ∘ R =", S_comp_R)
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> &
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Pr
S ∘ R = {(3, 7), (2, 6), (1, 5)}
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

4. Properties of Relations

Suppose the set A={1,2,3} and the relation R={(1,1),(2,2),(3,3),(1,2),(2,1)}.

Check whether the relation R is:

1. Reflexive
2. Symmetric
3. Antisymmetric
4. Transitive

Practical Assignment Steps:

• Create a Python function check_relation_properties(A, R) that checks the four properties above.

• Use if-else logic for each relation property.

• Display the result as True or False for each property.

• Add brief comments for each checking step.

The Source Code with output

```python
# Number 4
def check_relation_properties(A, R):
    # Check if R is reflexive: For all a in A, (a, a) must be in R
    is_reflexive = all((a, a) in R for a in A)

    # Check if R is symmetric: For all (a, b) in R, (b, a) must be in R
    is_symmetric = all((b, a) in R for (a, b) in R)

    # Check if R is antisymmetric: For all (a, b) in R, if a != b then (b, a) must NOT be in R
    is_antisymmetric = all((a != b) or ((b, a) not in R) for (a, b) in R)

    # Check if R is transitive: For all (a, b) and (b, c) in R, (a, c) must be in R
    is_transitive = True
    for (a, b) in R:
        for (b2, c) in R:
            if b == b2:
                if (a, c) not in R:
                    is_transitive = False
                    break
        if not is_transitive:
            break

    # Print the results
    print("Reflexive:", is_reflexive)
    print("Symmetric:", is_symmetric)
    print("Antisymmetric:", is_antisymmetric)
    print("Transitive:", is_transitive)

# Define the set A and relation R
A = {1, 2, 3}
R = {(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)}

# Check relation properties
check_relation_properties(A, R)

```

```
ts\Currents\Python> & C:/Users/USER/OneDrive/Documents/Currents
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> &
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Pr
Reflexive: True
Symmetric: True
Antisymmetric: False
Transitive: True
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

5. Equivalence Relations

An equivalence relation is a relation that is reflexive, symmetric, and transitive.

Suppose the set A={1,2,3,4} and the relation R={(1,1),(2,2),(3,3),(4,4),(1,3),(3,1)}. Check whether R is an equivalence relation.

Practical Assignment Steps:

• Reuse the function check_relation_properties(A, R) from the previous assignment.

• Check whether all properties (reflexive, symmetric, transitive) are True.

• If yes, display “R is an equivalence relation.”

• Add code comments and test with another relation that does not satisfy one of the properties.

The Source Code with output

```python

# Number 5
def check_relation_properties(A, R):
    # Check if R is reflexive: For all a in A, (a, a) must be in R
    is_reflexive = all((a, a) in R for a in A)

    # Check if R is symmetric: For all (a, b) in R, (b, a) must be in R
    is_symmetric = all((b, a) in R for (a, b) in R)

    # Check if R is antisymmetric: For all (a, b) in R, if a != b then (b, a) not in R
    # Not needed for equivalence, but included for completeness
    is_antisymmetric = all((a != b) or ((b, a) not in R) for (a, b) in R)

    # Check if R is transitive: For all (a, b) and (b, c) in R, (a, c) must be in R
    is_transitive = True
    for (a, b) in R:
        for (b2, c) in R:
            if b == b2:
                if (a, c) not in R:
                    is_transitive = False
                    break
        if not is_transitive:
            break

    return is_reflexive, is_symmetric, is_transitive

# Set A and relation R (candidate for equivalence relation)
A = {1, 2, 3, 4}
R = {(1, 1), (2, 2), (3, 3), (4, 4), (1, 3), (3, 1)}

# Check properties
reflexive, symmetric, transitive = check_relation_properties(A, R)

# Verify if R is an equivalence relation
if reflexive and symmetric and transitive:
    print("R is an equivalence relation.")
else:
    print("R is NOT an equivalence relation.")

# Test with another relation that does NOT satisfy one property
R2 = {(1, 1), (2, 2), (3, 3), (4, 4), (1, 3)}  # Missing (3, 1), so not symmetric
reflexive2, symmetric2, transitive2 = check_relation_properties(A, R2)

print("\nTest relation R2:")
if reflexive2 and symmetric2 and transitive2:
    print("R2 is an equivalence relation.")
else:
    print("R2 is NOT an equivalence relation.")
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> &
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Pr
R is an equivalence relation.

Test relation R2:
R2 is NOT an equivalence relation.
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

PRACTICUM Part 6: Applications of Partial Order Relations., Closure of Relations., N-ary Relations., Functions.

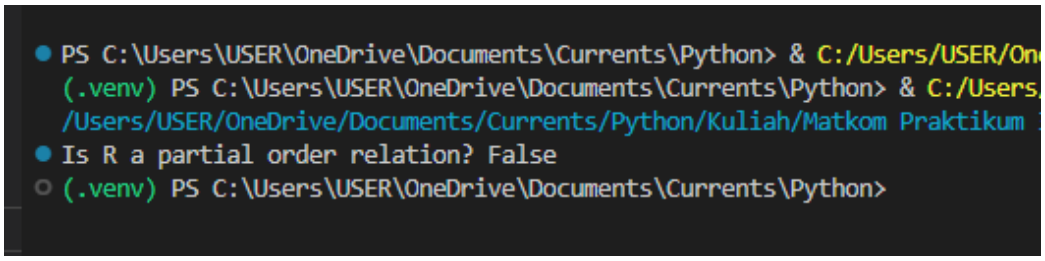
1. Partial Order Relations

A partial order relation is a relation that has the properties of reflexivity, antisymmetry, and transitivity. This relation is often represented using a Hasse diagram to show the hierarchical structure. Given the set A = {1, 2, 3} and the relation R = {(1,1), (2,2), (3,3), (1,2), (2,3)}. Determine whether R is a partial order relation.

The Source Code with output

Kuliah > Matkom Praktikum 3 > part6.py > ...

```python
# Number 1
# Defining the set A and relation R
A = {1, 2, 3}
R = {(1, 1), (2, 2), (3, 3), (1, 2), (2, 3)}

# Check the reflexive property
is_reflexive = all((a, a) in R for a in A)

# Check the antisymmetric property:
is_antisymmetric = all((b, a) not in R for (a, b) in R if a != b)

# Check the transitive property:
is_transitive = all(
    (a, c) in R
    for (a, b) in R
    for (b2, c) in R
    if b == b2
)

# Determine whether R is a partial order relation
is_partial_order = is_reflexive and is_antisymmetric and is_transitive

print("Is R a partial order relation?", is_partial_order)
```



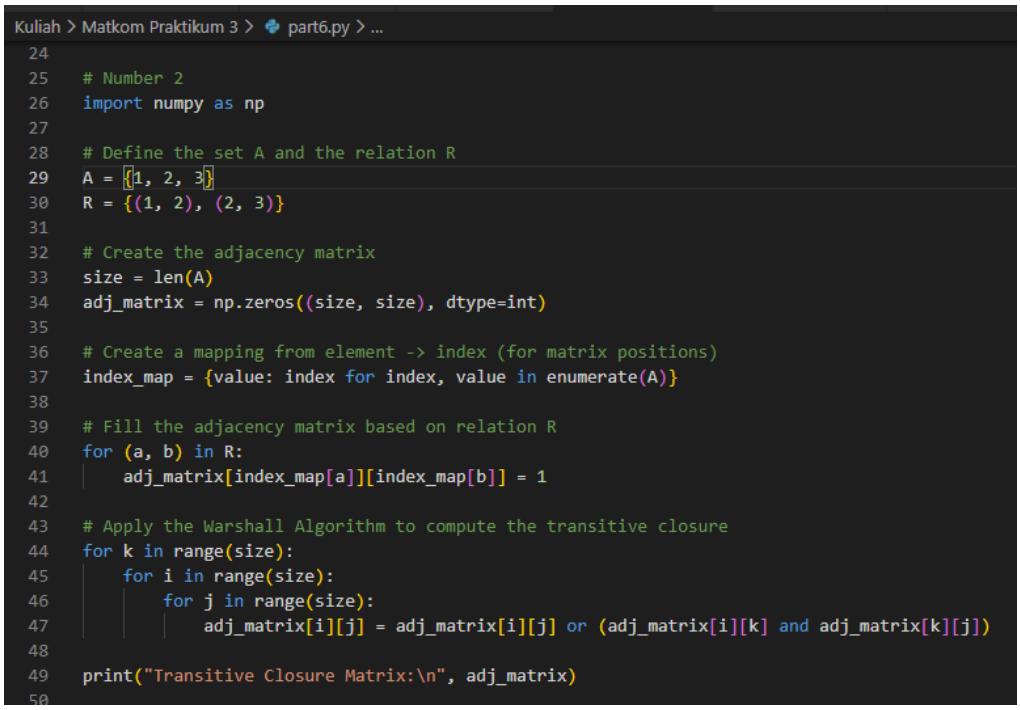
The Analysis

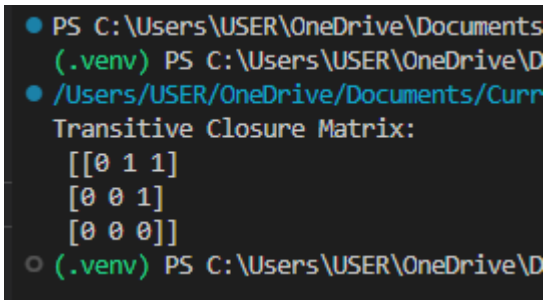
The code determines whether a given relation R on the set A={1,2,3} forms a partial order by checking the three required properties: reflexivity, antisymmetry, and transitivity. Reflexivity is verified by confirming that every element aaa in A includes a self-pair (a,a) in the relation, while antisymmetry is checked by ensuring that for any pair (a,b) in R, the reversed pair (b,a) is not present unless a=b, preventing mutual two-way relations except on identical elements. Transitivity is validated by examining all pair chains (a,b) and (b,c) within the relation and ensuring the implied pair (a,c) is also in the relation, which guarantees proper ordering consistency. After evaluating these conditions, the code combines the three Boolean results to determine whether RRR satisfies all criteria of a partial order relation and prints the conclusion, offering a clear and systematic assessment of the structure of the relation.

2. **Closure of Relations**

The closure of a relation is the process of adding elements to a relation so that it satisfies certain properties, such as reflexivity or transitivity. For example, the transitive closure will add the necessary pairs to make the relation transitive. Given the relation R = {(1,2), (2,3)} on the set A = {1,2,3}. Determine the transitive closure of R.

The Source Code with output





The Analysis

The code computes the transitive closure of a relation R on the set A={1,2,3}by first converting the relation into an adjacency matrix representation and then applying the Warshall algorithm to identify all indirect connections. It begins by initializing a zero matrix sized according to the number of elements in A and then uses an index mapping to correctly position each pair (a,b) from the relation into the matrix. Once the adjacency matrix is filled, the Warshall algorithm iterates through all possible intermediate nodes k, updating the matrix so that a path from i to j exists if either a direct relation was already present or if i can reach j through an intermediate element. The final printed matrix represents the transitive closure, clearly showing all direct and indirect reachabilities within the relation, making it a

powerful tool for analyzing connectivity and structure in discrete mathematics and graph theory.

3. N-ary Relations

An n-ary relation is a relation between more than two sets. This relation connects multiple entities at once, for example, the relationship between students, subjects, and grades.

Suppose a 3-ary relation R that connects students, subjects, and grades: R = {("Deny","Mathematics",90),("Bob","Science",85)}.
Display each element of R and explain the meaning of each pair.

The Source Code with output

```
50
51  # Number 3
52  # Defining a 3-ary relation R (student, subject, score)
53  R = [("Deny", "Mathematics", 90),
54       ("Bob", "Science", 85)]
55
56  # Display each element in R
57  for student, subject, score in R:
58      print(f"Student: {student}, Subject: {subject}, Score: {score}")
59
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/Users
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> &
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Pr
Student: Deny, Subject: Mathematics, Score: 90
Student: Bob, Subject: Science, Score: 85
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

The code defines a 3-ary relation R, where each tuple represents a student, a subject, and the corresponding score, effectively modeling a relationship involving three attributes instead of just ordered pairs. By storing elements like ("Deny", "Mathematics", 90) and ("Bob", "Science", 85), the relation captures richer information that can be used to describe academic performance across different subjects. The for loop then iterates through each tuple, unpacking its three components student name, subject, and score and printing them in a clear, human-

readable format. This simple structure demonstrates how higher-arity relations can be represented and processed in Python, making it easier to handle multidimensional data in discrete mathematics or database-related tasks.

4. Functions

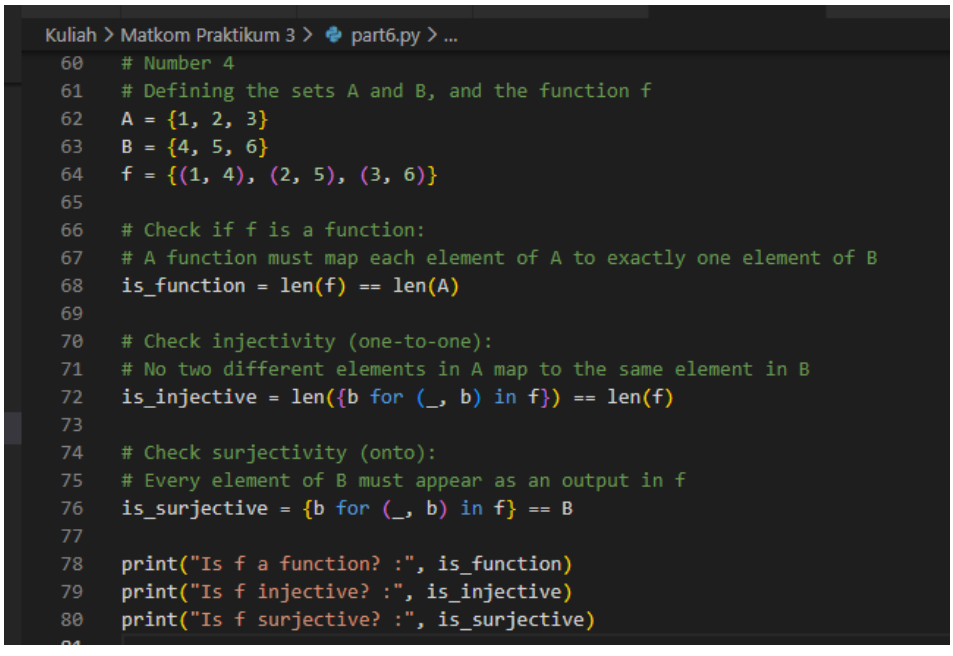
Brief Theory:

A function f from set A to set B is a relation where each element in A is related to exactly one element in B. Function characteristics can be injective (one-to-one) or surjective (onto).

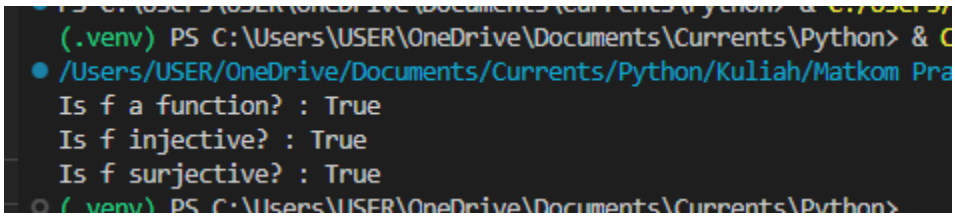
Given sets A = {1, 2, 3} and B = {4, 5, 6}, define function f as f = {(1,4), (2,5), (3,6)}.

Check whether this function is injective and surjective.

The Source Code with output



```
Kuliah > Matkom Praktikum 3 > part6.py > ...
60  # Number 4
61  # Defining the sets A and B, and the function f
62  A = {1, 2, 3}
63  B = {4, 5, 6}
64  f = {(1, 4), (2, 5), (3, 6)}
65
66  # Check if f is a function:
67  # A function must map each element of A to exactly one element of B
68  is_function = len(f) == len(A)
69
70  # Check injectivity (one-to-one):
71  # No two different elements in A map to the same element in B
72  is_injective = len({b for (_, b) in f}) == len(f)
73
74  # Check surjectivity (onto):
75  # Every element of B must appear as an output in f
76  is_surjective = {b for (_, b) in f} == B
77
78  print("Is f a function? :", is_function)
79  print("Is f injective? :", is_injective)
80  print("Is f surjective? :", is_surjective)
```



```
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom Pra
Is f a function? : True
Is f injective? : True
Is f surjective? : True
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

The Analysis

The code evaluates whether a given relation f from set A={1,2,3} to set B={4,5,6} qualifies as a proper mathematical function, and then checks if it is injective and surjective. First, it verifies the function requirement by comparing the number of ordered pairs in f with the number of elements in A, ensuring that every element in A maps to exactly one element in B. Injectivity is checked by collecting all output values b in a set and confirming that the count of unique outputs matches the number of input-output pairs, guaranteeing that no two distinct elements in A map to the same element in B. Surjectivity is tested by comparing the set of all output values in f with the entire codomain B, making sure that every element of B appears as an output. Finally, the program prints the results of all three checks, clearly indicating whether f is a function, injective, and surjective, thus providing a complete characterization of the function's mapping properties.

SUMMARY

1. Checking Partial Order Relations by verifying the reflexive, antisymmetric, and transitive properties.
2. Calculating Relation Closure (transitive closure) using Warshall's Algorithm.
3. Representing n-ary relations that connect more than two sets.
4. Identifying properties of Functions and verifying whether a function is injective or surjective.

PRACTICUM Part 7: Practice Exercises of Partial Order Relations., Closure of Relations., N-ary Relations., Functions.

1. **Partial Order Relations**

Given: Set A={1,2,3,4} and relation R={(1,1),(2,2),(3,3),(4,4),(1,2),(2,3),(1,3)}.
Determine whether R is a partial order on A.

Practical Assignment Steps:

- Create a Python function is_partial_order(A, R) to check whether R is reflexive, antisymmetric, and transitive.
- Use logic to verify each property in the code.
- Display True/False results for each property and the final conclusion.
- Add brief comments in each section of the code.

The Source Code with output

```python
# Number 1
# Function to check whether a relation R on set A is a partial order
def is_partial_order(A, R):
    # Convert to sets for easy membership tests
    A = set(A)
    R = set(R)

    # Check reflexivity: every (a, a) must be in R
    reflexive = all((a, a) in R for a in A)

    # Check antisymmetry: if (a, b) and (b, a) in R, then a == b
    antisymmetric = True
    for a, b in R:
        if (b, a) in R and a != b:
            antisymmetric = False
            break

    # Check transitivity: if (a, b) and (b, c) in R, then (a, c) must be in R
    transitive = True
    for a, b in R:
        for x, c in R:
            if b == x and (a, c) not in R:
                transitive = False
                break
        if not transitive:
            break

    # Display results
    print("Reflexive:", reflexive)
    print("Antisymmetric:", antisymmetric)
    print("Transitive:", transitive)
    print("Is partial order:", reflexive and antisymmetric and transitive)

    return reflexive and antisymmetric and transitive

```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Python> & C:/User
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python> &
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom
Reflexive: True
Antisymmetric: True
Transitive: True
Is partial order: True
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

2. Closure of Relations

Given: Set A={1,2,3} and relation R={(1,2),(2,3)}.

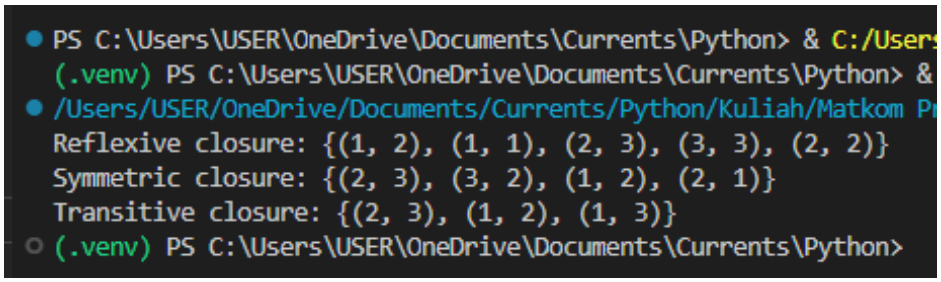
Determine the reflexive, symmetric, and transitive closures of relation R.

Practical Assignment Steps:

- Create three Python functions:
  - reflexive_closure(A, R)
  - symmetric_closure(R)
  - transitive_closure(R)
- Add the necessary elements to satisfy each property.
- Use loops to add missing pairs.
- Display the result of each closure as a set of pairs.
- Add code comments to explain the logic of forming the closures.

The Source Code with output

```python
# Number 2
# Build the reflexive closure of R on set A
def reflexive_closure(A, R):
    R = set(R)
    for a in A:
        # Add missing (a,a) pairs
        if (a, a) not in R:
            R.add((a, a))
    return R

# Build the symmetric closure of R
def symmetric_closure(R):
    R = set(R)
    for a, b in list(R):
        # Add (b,a) if missing
        if (b, a) not in R:
            R.add((b, a))
    return R

# Build the transitive closure of R
def transitive_closure(R):
    R = set(R)
    added = True

    # Keep adding (a,c) whenever (a,b) and (b,c) are present
    while added:
        added = False
        new_pairs = set()
        for a, b in R:
            for x, c in R:
                if b == x and (a, c) not in R:
                    new_pairs.add((a, c))
        if new_pairs:
            R |= new_pairs
            added = True
    return R

# Example usage
A = {1, 2, 3}
R = {(1,2), (2,3)}

print("Reflexive closure:", reflexive_closure(A, R))
print("Symmetric closure:", symmetric_closure(R))
print("Transitive closure:", transitive_closure(R))

```



3. N-ary Relations

Given: A ternary (3-ary) relation between Students, Courses, and Grades:

R={("Andi","Matematika",90),("Budi","Fisika",85),("Citra","Kimia",88)}.

Practical Assignment Steps:

- Create a Python function display_nary_relation(R) to display the relation in a tabular format.
- Use a loop to print each tuple in a formatted way.
- Add columns for "Mahasiswa" (Student), "Mata Kuliah" (Course), and "Nilai" (Grade).
- Test by adding some new data to the relation.
- Add brief comments in each section of the code.

The Source Code with output

```python
# Number 3
# Function to display a 3-ary relation in a simple table
def display_nary_relation(R):
    # Print header
    print("Mahasiswa\tMata Kuliah\tNilai")
    print("----------------------------------------")

    # Loop through each tuple and print in formatted columns
    for student, course, grade in R:
        print(f"{student}\t{course}\t{grade}")

# Original relation
R = {
    ("Andi", "Matematika", 90),
    ("Budi", "Fisika", 85),
    ("Citra", "Kimia", 88)
}

# Add a few new data points for testing
R.add(("Dina", "Biologi", 92))
R.add(("Eko", "Matematika", 78))

# Display the full relation
display_nary_relation(R)

```

```
/Users/USER/OneDrive/Documents/Currents/Python/Kuliah/Matkom P
Mahasiswa       Mata Kuliah     Nilai
----------------------------------------
Budi    Fisika  85
Andi    Matematika      90
Eko     Matematika      78
Dina    Biologi 92
Citra   Kimia   88
(.venv) PS C:\Users\USER\OneDrive\Documents\Currents\Python>
```

4. Functions

Given: Sets A={1,2,3,4} and B={2,4,6,8}.

Define a function f:A→B with the rule f(x)=2x.

Practical Assignment Steps:

- Create a Python function is_function(A, B, R) to check whether a relation R is a function.
- Use the relation R={(1,2),(2,4),(3,6),(4,8)}.
- Ensure each element of A has exactly one partner in B.
- Add logic to reject relations that do not satisfy the requirements of a function.
- Print the result of the check (True or False) and display the function relation.

The Source Code with output

```python
# Number 4
# Check whether a relation R from A to B is a function
def is_function(A, B, R):
    A = set(A)
    B = set(B)
    R = set(R)

    # Track mappings from elements of A
    mapping = {}

    for a, b in R:
        # Reject if a is not in A or b is not in B
        if a not in A or b not in B:
            return False

        # Reject if a maps to more than one value
        if a in mapping and mapping[a] != b:
            return False

        mapping[a] = b

    # Every element of A must have exactly one output
    if len(mapping) != len(A):
        return False

    return True


# Given sets and relation
A = {1, 2, 3, 4}
B = {2, 4, 6, 8}
R = {(1,2),(2,4),(3,6),(4,8)}

# Display result
print("Relation R:", R)
print("Is R a function from A to B?", is_function(A, B, R))
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Pyth
(.venv) PS C:\Users\USER\OneDrive\Documents\Curre
/Users/USER/OneDrive/Documents/Currents/Python/Ku
Relation R: {(2, 4), (1, 2), (4, 8), (3, 6)}
Is R a function from A to B? True
(.venv) PS C:\Users\USER\OneDrive\Documents\Curre
```

PRACTICUM Part 8: Applications of Inverse Functions., Composition of Functions., Some Special Functions., Recursive Functions.

1. Inverse Functions

An inverse function is a function that returns a value back to its original element in a one-to-one relation. For example, if f(x) = y, then the inverse function of f, written f−1(y) = x. Given the function f(x) = 2x+3, determine its inverse function and implement it in Python.

The Source Code with output

```
Kuliah > Matkom Praktikum 3 > part8.py > ...
1   # Number 1
2   def f(x):
3       return 2 * x + 3
4
5   def f_inverse(y):
6       return (y - 3) / 2
7
8   x = 10
9   y = f(x)
10  print("f(x) =", y)
11  print("f_inverse(y) =", f_inverse(y))
```

```
PS C:\Users\USER\OneDr
(.venv) PS C:\Users\US
/Users/USER/OneDrive/D
f(x) = 23
f_inverse(y) = 10.0
(.venv) PS C:\Users\US
```

The Analysis

The code creates a function that takes a number, multiplies it by two, and then adds three. It also creates an opposite function that can reverse this process. When the program starts, it uses the number ten as input and calculates the result of the function, which becomes twenty-three. Then it sends that result into the opposite

function, which brings the number back to ten. This shows that the second function correctly cancels or reverses the effect of the first one.

2. Composition of Functions

Function composition is the combination of two functions where the output of one function becomes the input for the other. If f and g are functions, then the composition (f∘g) (x) = f(g(x)). Given two functions f(x) = x+2 and g(x) =3x, determine the composition (f∘g)(x).

The Source Code with output

```
13  # Number 2
14  def f(x):
15      return x + 2
16
17  def g(x):
18      return 3 * x
19
20  def f_g(x):
21      return f(g(x))
22
23  x = 4
24  print("f(g(x)) =", f_g(x))
25
```

```
PS C:\Users\USER\OneDrive\Do
(.venv) PS C:\Users\USER\One
/Users/USER/OneDrive/Documer
f(g(x)) = 14
(.venv) PS C:\Users\USER\One
```

The Analysis

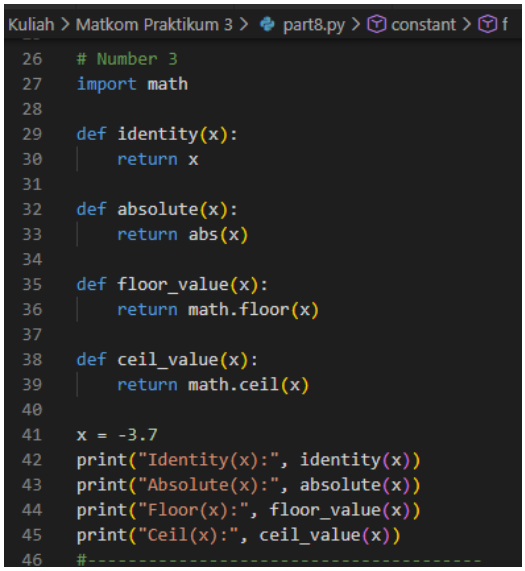
The code defines two functions: one adds two to any number, and the other multiplies a number by three. A third function combines them by first using the multiply function and then using the add function on the result. When the input is four, the program first multiplies four by three to get twelve, and then adds two to

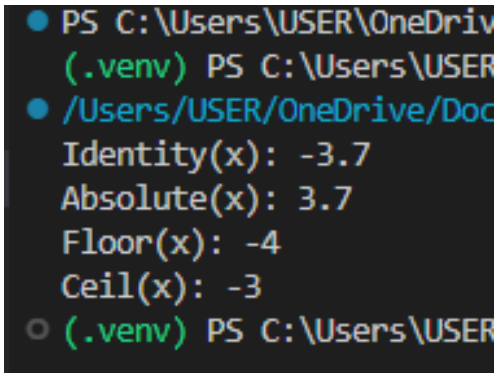
get fourteen. The final print statement shows the result of this combined process in a clear and simple way.

3. Some Special Functions

Some special functions in mathematics that are commonly used include the identity function, absolute value function, floor function, and ceiling function. Implement the identity function, absolute value function, floor function, and ceiling function using Python.

The Source Code with output





The Analysis

The code shows four small functions that each do something different to a number. The first one simply returns the number as it is, without changing anything. The second one gives the positive version of the number even if the input is negative. The third one rounds the number down to the nearest whole number, while the

fourth one rounds it up to the nearest whole number. When the input is negative three point seven, the program prints the original number, the positive version, the rounded-down result, and the rounded-up result, helping you see clearly how each function behaves.

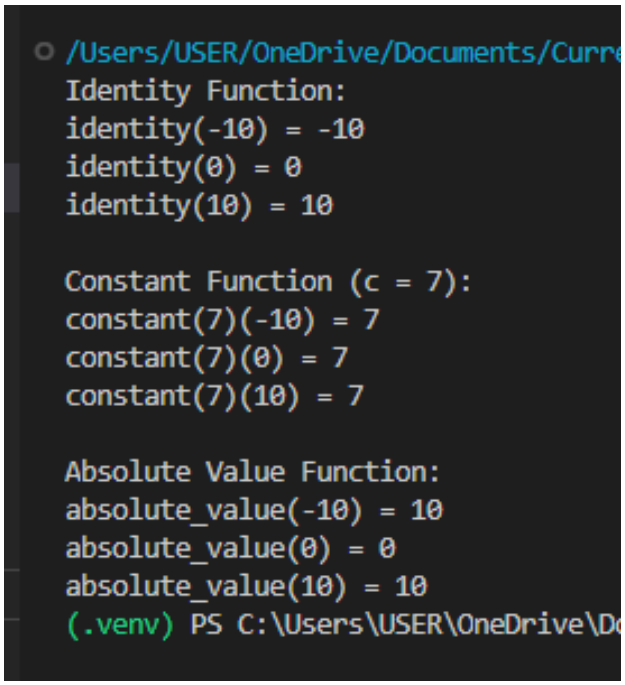
Other Special Functions

1. Identity Function
   A function that returns the input value unchanged, i.e., f(x) = x.
2. Constant Function
   A function that always returns a fixed value, regardless of the input, i.e., f(x) = c, where c is a constant.
3. Absolute Value Function
   A function that returns the positive value of the input, i.e., f(x) = |x|.

Task:
Implement the above three special functions using Python, and test them with several input values.

The Source Code with output

```
46  #---------------------------------------
47  def identity(x):
48      return x
49
50  def constant(c):
51      def f(x):
52          return c
53      return f
54
55  def absolute_value(x):
56      return abs(x)
57
58  x_values = [-10, 0, 10]
59
60  print("Identity Function:")
61  for x in x_values:
62      print(f"identity({x}) =", identity(x))
63
64  print("\nConstant Function (c = 7):")
65  constant_func = constant(7)
66  for x in x_values:
67      print(f"constant(7)({x}) =", constant_func(x))
68
69  print("\nAbsolute Value Function:")
70  for x in x_values:
71      print(f"absolute_value({x}) =", absolute_value(x))
72
```



The Analysis

The code shows three different types of functions and tests them on the numbers negative ten, zero, and ten. The first function simply returns the number exactly as it is, without changing anything. The second function always returns the same value, which in this case is seven, no matter what input you give it. The third function returns the positive form of the number, so negative values become positive while positive values stay the same. The program prints the results of each function for all three test numbers, making it easy to see how each type of function behaves.

4. **Recursive Functions**

A recursive function is a function that calls itself. A classic example of a recursive function is the computation of the factorial and the Fibonacci sequence. Implement the factorial and Fibonacci functions using recursion.

The Source Code with output

```python
# Number 4
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = 9
print("Factorial(n):", factorial(n))
print("Fibonacci(n):", fibonacci(n))

```

```
PS C:\Users\USER\OneDrive\D
(.venv) PS C:\Users\USER\On
/Users/USER/OneDrive/Docume
Factorial(n): 362880
Fibonacci(n): 34
(.venv) PS C:\Users\USER\On
```

The Analysis

The code has two functions that each work by repeating the same process over and over until they reach a stopping point. The first function calculates the result of multiplying all whole numbers down to one, and it stops when the number reaches zero. The second function creates a number by adding the two previous results in the sequence, and it stops when the number is zero or one. The program uses the number nine as input, prints the result of the first function, and then prints the result of the second one. This shows how both functions grow their results step by step using smaller versions of themselves.

CONCLUSIONS

In this module, we have studied and implemented:

1. Inverse functions to find the reverse value of a function.
2. Function composition to combine two functions into one.

3. Special functions such as identity, absolute, floor, and ceiling functions.
4. Recursive functions such as factorial calculation and the Fibonacci sequence.

PRACTICUM Part 9: Practice Exercises of Inverse Functions., Composition of Functions., Some Special Functions., Recursive Functions.

1. Inverse Functions

Suppose a function f: A→B is defined by f(x) = 2x+1, where A = {1,2,3,4}.

Determine its inverse function f−1.

Practical Assignment Steps:

- Create a Python function inverse_function (A, f) to determine the inverse function of f.
- Use a dictionary to map pairs (x,f(x)).
- Swap the positions of keys and values to form the inverse function.
- Print the results of f and f−1 in the form of ordered pairs.
- Add brief comments at each step of the code.

The Source Code with output

```python
# Number 1
# Determine the inverse of a function represented as a dictionary
def inverse_function(A, f):
    # f is a dict mapping x -> f(x)
    inverse_f = {}

    # Swap keys and values to get the inverse mapping
    for x in A:
        y = f[x]
        inverse_f[y] = x  # store f⁻¹(y) = x

    return inverse_f


# Build the function f(x) = 2x + 1 over A
A = {1, 2, 3, 4}
f = {x: 2*x + 1 for x in A}

# Compute its inverse
inv_f = inverse_function(A, f)

# Display the pairs of f and f⁻¹
print("f =", set((x, f[x]) for x in A))
print("f_inverse =", set((y, inv_f[y]) for y in inv_f))
```

```
PS C:\Users\USER\OneDrive\Documents\Currents\Pyt
(.venv) PS C:\Users\USER\OneDrive\Documents\Curr
/Users/USER/OneDrive/Documents/Currents/Python/k
f = {(4, 9), (3, 7), (2, 5), (1, 3)}
f_inverse = {(3, 1), (9, 4), (7, 3), (5, 2)}
(.venv) PS C:\Users\USER\OneDrive\Documents\Curr
```

2. Composition of Functions

Given two functions:

f(x)=2x+3

g(x)=x2

Determine the results of the function compositions (f∘g)(x) and (g∘f)(x).

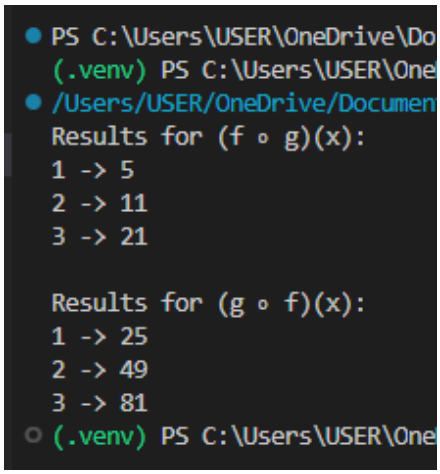
Practical Assignment Steps:

- Create two Python functions f(x) and g(x) according to the definitions above.

- Create a function compose (f, g, x) that returns the value of the composition f(g(x)).
- Calculate and display the values of (f∘g)(x) and (g∘f)(x) for several values of x (for example, x=1,2,3).
- Add code comments to explain the function composition process.

The Source Code with output

```python
# Number 2
# Define f(x) = 2x + 3
def f(x):
    return 2*x + 3

# Define g(x) = x^2
def g(x):
    return x**2

# Composition function: compute f(g(x))
def compose(f, g, x):
    # First apply g to x, then f to the result
    return f(g(x))

# Test values
values = [1, 2, 3]

print("Results for (f ∘ g)(x):")
for x in values:
    print(x, "->", compose(f, g, x))

print("\nResults for (g ∘ f)(x):")
for x in values:
    # Now compute g(f(x))
    print(x, "->", g(f(x)))
```



3. Some Special Functions

Study the following types of functions:

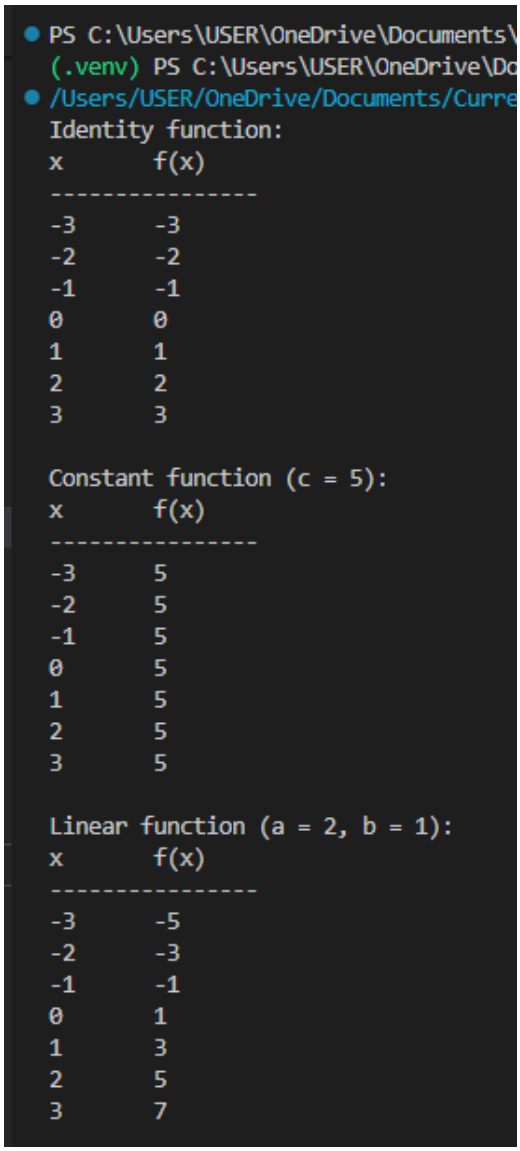
- Identity function: f(x) = x
- Constant function: f(x) = c
- Linear function: f(x) = ax+b
- Quadratic function: f(x) = ax2+bx+c

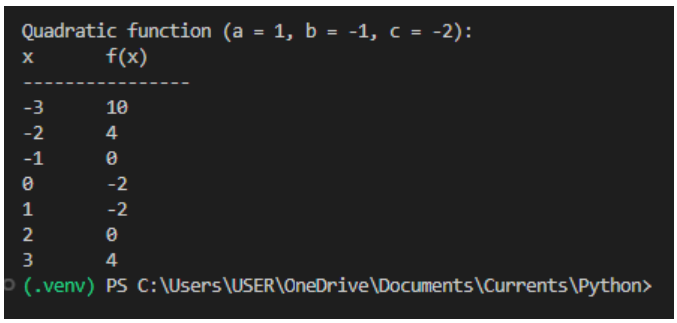
Practical Assignment Steps:

- Implement each function in Python using the mathematical definitions.
- Create a Python function to display function values for x = −3 to 3.
- Display the results in a table format with x and f(x).
- Add comments in the code to explain the differences among each function.
- Test changes in the values of a, b, and c to observe their effect on the results.

The Source Code with output

```python
# Number 3
# Identity function: returns the input unchanged
def identity(x):
    return x

# Constant function: always returns the same value c
def constant(x, c=5):
    return c

# Linear function: f(x) = a*x + b
def linear(x, a=2, b=1):
    return a*x + b

# Quadratic function: f(x) = a*x^2 + b*x + c
def quadratic(x, a=1, b=0, c=0):
    return a*x**2 + b*x + c


# Display function values for x = -3 to 3
def display_table(func, *params):
    print("x\tf(x)")
    print("----------------")
    for x in range(-3, 4):
        print(f"{x}\t{func(x, *params) if params else func(x)}")

# Demonstrations
print("Identity function:")
display_table(identity)

print("\nConstant function (c = 5):")
display_table(constant, 5)

print("\nLinear function (a = 2, b = 1):")
display_table(linear, 2, 1)

print("\nQuadratic function (a = 1, b = -1, c = -2):")
display_table(quadratic, 1, -1, -2)

```





4. Recursive Functions

Create a program to compute the factorial using a recursive function.

Practical Assignment Steps:

• Define a Python function factorial(n) that computes n! using recursion.

• Use a base case: if n == 0 or n == 1, return 1.

• Otherwise, return n * factorial (n - 1).

• Test the function with several values of n (for example, 3, 5, and 7).

• Add comments in each part of the code to explain the recursion process.

The Source Code with output

```python
# Number 4
# Identity function: returns the input x exactly
def identity(x):
    return x

# Constant function: always returns the same fixed value c
def constant(x, c=5):
    return c

# Linear function: f(x) = a*x + b
# The parameters a and b control slope and vertical shift
def linear(x, a=1, b=0):
    return a*x + b

# Quadratic function: f(x) = a*x^2 + b*x + c
# The parameter a controls curvature, b the tilt, c the vertical shift
def quadratic(x, a=1, b=0, c=0):
    return a*x**2 + b*x + c


# Function to display x and f(x) values in table form
def display_table(func, *params):
    print("x\tf(x)")
    print("----------------")
    for x in range(-3, 4):
        # If the function has additional parameters (a, b, c), pass them
        print(f"{x}\t{func(x, *params) if params else func(x)}")
    print()


# Test outputs for each type of function
print("Identity function:")
display_table(identity)

print("Constant function (c = 5):")
display_table(constant, 5)

print("Linear function (a = 2, b = -1):")
display_table(linear, 2, -1)

print("Quadratic function (a = 1, b = -2, c = -3):")
display_table(quadratic, 1, -2, -3)
```

```
(.venv) PS C:\Users\USER\OneDrive\Docu
/Users/USER/OneDrive/Documents/Current
Identity function:
x       f(x)
---------------
-3      -3
-2      -2
-1      -1
0       0
1       1
2       2
3       3

Constant function (c = 5):
x       f(x)
---------------
-3      5
-2      5
-1      5
0       5
1       5
2       5
3       5

Linear function (a = 2, b = -1):
x       f(x)
---------------
-3      -7
-2      -5
-1      -3
0       -1
1       1
2       3
3       5
```

```
Quadratic function (a = 1, b = -2, c = -3):
x       f(x)
---------------
-3      12
-2      5
-1      0
0       -3
1       -4
2       -3
3       0
```